網路流

網路流以容量與流量守恆描述資源如何從來源傳到匯點。殘餘網路記錄增加流量與撤銷既有決策的可能性。

本章依序研究 **增廣路徑、最大流最小割與 Edmonds–Karp → push–relabel → 匹配與下界建模**。

前置知識為圖、廣度優先搜尋、不變量與漸進分析。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從流量到證明與建模

問題	要檢查的條件
流量是否合法？	容量限制與中間頂點的流量守恆。
如何改進或證明最優？	殘餘路徑、割容量與最大流最小割定理。
局部操作如何收斂？	前流、頂點高度與 push–relabel 不變量。
如何表示新限制？	匹配、頂點容量與有下界的可行流。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A01 | 網路流要交付可驗證的最優證書

給定有向網路 G=(V,E)、源點 s、匯點 t 與有限非負容量 c。求可行流 f 的最大淨流出量。

本節由原筆記六點例推進，最後同時交付值為11的流與容量11的割。來源為 [CLRS26] §26.1–26.2、[MIT04-20]、[MIT04-21]、[NOTE] 38–40頁。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A02 | 容量限制與守恆是不同條件

每條原邊 e 滿足 0≤f(e)≤c(e)。每個 v∈{s,t} 滿足 Σ入邊f=Σ出邊f。

定義 |f|=Σ(s,v)f(s,v)−Σ(v,s)f(v,s)。若源點有入邊，不能只算流出量。對所有內點求和，源點淨流出恰為匯點淨流入。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A03 | 固定流模型，避免反平行邊混淆

本節 f 是每條原邊的非負流，不採 MIT04 的反對稱淨流記法。原例無反平行原邊。

若同時有 u→v 與 v→u，每條原邊各有自己的正向與反向殘餘邊，以 edge_id 配對。u→v 方向可同時含 c(u,v)−f(u,v) 的加流通道與 f(v,u) 的取消通道。可合併容量，但更新時必須分攤。另一做法是把一條原邊拆成 u→q→v。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A04 | 殘餘邊是允許的流量修改

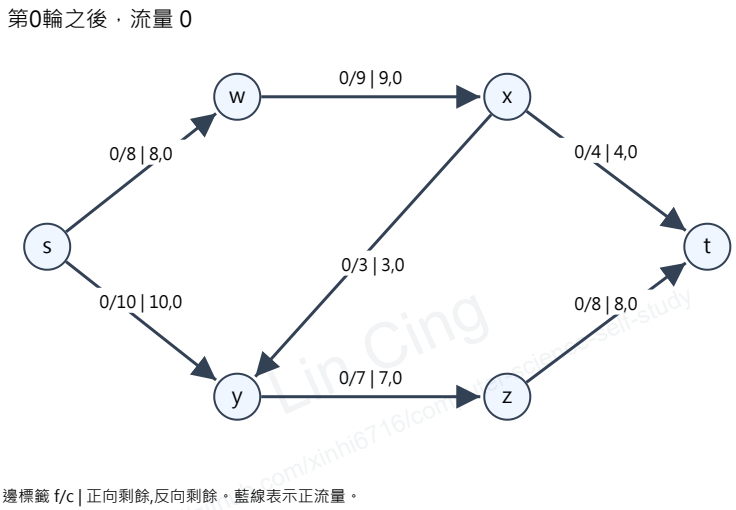
原邊 u→v 的正向殘餘為 c−f，反向 v→u 的殘餘為 f。只把正殘餘放進 G_f。

沿反向邊送 δ，意思是原邊 f 減 δ。它不是新增一條實體運輸線，也不受不存在的原邊容量0阻擋。增廣前必須保留反向邊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A05 | 六點網路與零流

容量依序為 sw=8、sy=10、wx=9、xy=3、yz=7、xt=4、zt=8。所有邊起始 f=0。



圖中標籤為 f/c | 正向剩餘,反向剩餘。箭頭是原邊方向，反向剩餘由標籤給出。

A09-A06 | 增廣保留可行性並增加流值

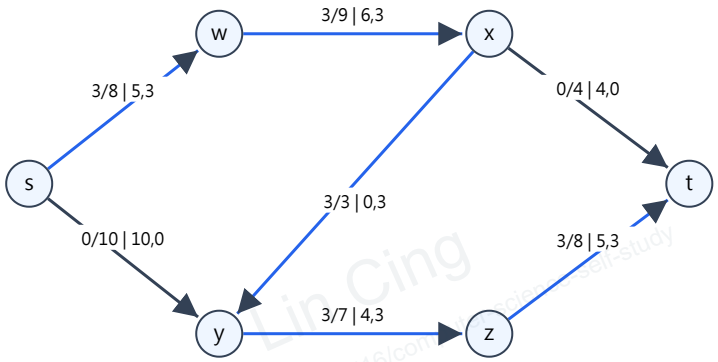
選 G_f 中簡單 s→t 路 P。δ=min_{e∈P}r(e)>0。正向原邊加δ，反向原邊減δ。

容量由 δ≤r 保證。路徑內點恰有一單位入流修改與一單位出流修改，淨平衡仍為0。s 淨流出增 δ，t 淨流入增δ，因此新流值為 |f|+δ。至少一條殘餘邊飽和。

A09-A07 | 第1輪選一條較長路

選 P_1=s→w→x→y→z→t，殘餘為8,9,3,7,8，所以 δ_1=3。流值3。

第1輪之後，流量 3



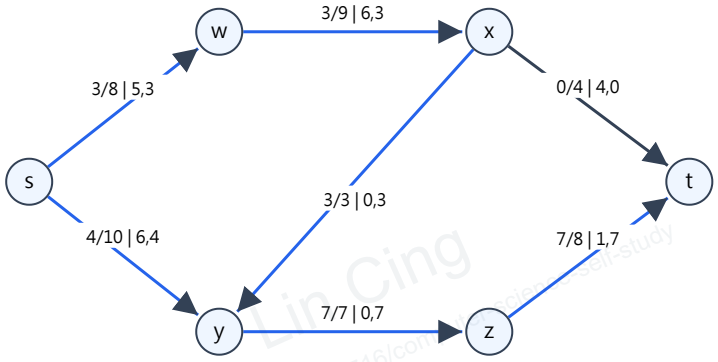
邊標籤 f/c | 正向剩餘,反向剩餘。藍線表示正流量。

邊 xy 已飽和，卻產生 y→x 的反向殘餘3。這是一般 Ford–Fulkerson 的合法選路，並非 Edmonds–Karp。

A09-A08 | 第2輪先填滿 yz

選 P_2=s→y→z→t，殘餘10,4,5，所以 δ_2=4。流值7。

第2輪之後，流量 7

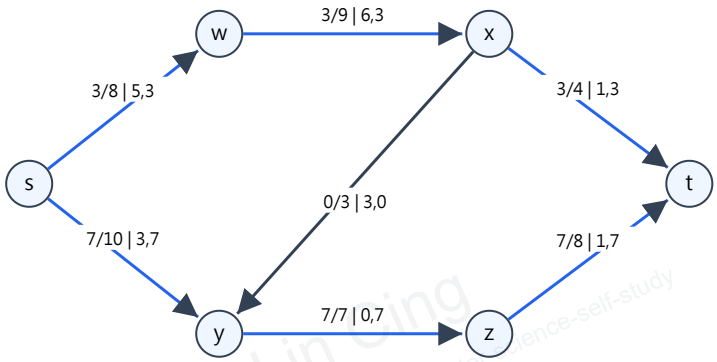


邊標籤 f/c | 正向剩餘,反向剩餘。藍線表示正流量。

此時 x 的入流3=出流3；y 的入流3+4=出流7。單看原圖從 s 到 t 的未飽和路，會錯過下一輪的取消。

A09-A09 | 第3輪使用逆邊取消

第3輪之後，流量 10



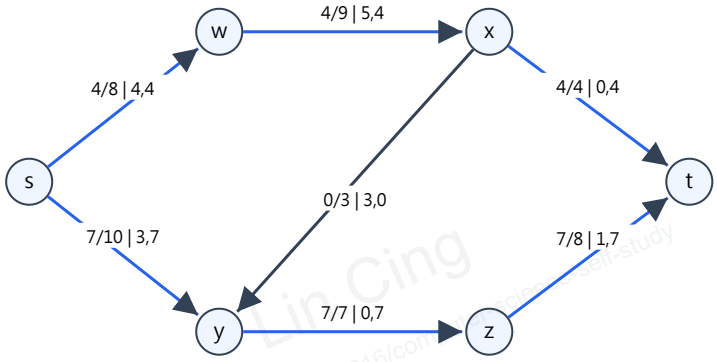
邊標籤 f/c | 正向剩餘,反向剩餘。藍線表示正流量。

選 P_3=s→y→x→t，殘餘6,3,4，所以 δ_3=3。y→x 是 xy 的反向邊，令 f(xy) 由3降為0。流值10。取消後 x 的入流3=出流3；y 的入流7=出流7。流值增加，單邊流量可以減少。

A09-A10 | 第4輪與所有內點的守恆驗算

選 P_4=s→w→x→t，殘餘5,6,1，所以 δ_4=1。最終流值11。

第4輪之後，流量 11



邊標籤 f/c | 正向剩餘,反向剩餘。藍線表示正流量。

最終 w 為4入4出，x 為4入4出，y 為7入7出，z 為7入7出。源點4+7=11，匯點4+7=11。所有邊流依 sw,sy,wx,xy,yz,xt,zt 為 (4,7,4,0,7,4,7)。

A09-A11 | 每一輪都可逐項重算

輪次	sw	sy	wx	xy	yz	xt	zt	值
0	0	0	0	0	0	0	0	0
1	3	0	3	3	3	0	3	3
2	3	4	3	3	7	0	7	7
3	3	7	3	0	7	3	7	10
4	4	7	4	0	7	4	7	11

每列驗 w:sw=wx；x:wx=xy+xt；y:sy+xy=yz；z:yz=zt。check.py 對全部輪次與16種 s–t 割逐一驗算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A12 | 割的容量與淨流不能混算

割 (S,T) 滿足 s∈S、t∈T、T=V\S。

c(S,T)=Σ_{u∈S,v∈T}c(u,v)，只加 S→T 原邊容量。f(S,T)=Σ_{S→T}f−Σ_{T→S}f。

將 S 內頂點的出流減入流加總，內邊兩兩消去，內點守恆只留下 s，所以 f(S,T)=|f|。再用容量與非負性得 |f|≤c(S,T)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A13 | 最優流與最小割的雙向證明

若存在割使 |f|=c(S,T)，任何可行流 g 都有 |g|≤c(S,T)=|f|，故 f 最大且此割最小。

反方向，若 f 最大，存在增廣路就能增加流值，矛盾，故沒有增廣路。令 S 為 G_f 中從s可達的點。t不可達，所以形成合法割。每條 S→T 原邊飽和，否則可跨割；每條 T→S 原邊流為0，否則其反向殘餘可跨割。因此 |f|=f(S,T)=c(S,T)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A14 | 例子的停止證書

最終 G_f 從 s 可達 S={s,w,x,y}，T={z,t}。sw,sy,wx,xy 的正殘餘維持這些可達性，但 yz、xt 正殘餘均為0。

割容量 c(S,T)=c(yz)+c(xt)=7+4=11。沒有 T→S 正流。已構造可行流11且任一流≤11，故最大流=最小割=11。不能把 zt 的剩餘1誤當可增廣量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A15 | Ford–Fulkerson 的整數終止性

```
f[e] = 0 for each original edge e
while find_path(G_f, s, t) returns P:
    delta = min(residual[arc] for arc in P)
    update each paired original edge along P by +delta or -delta
return f, reachable(G_f, s)
```

初始化0；只要有增廣路就依瓶頸增廣。整數容量使每輪殘餘與δ為整數，δ≥1。流值受有限割容量限制，因此至多 F*=|f*| 輪。

以鄰接串列找路並更新 O(E) 時，時間 O(E(1+F*))，通常寫 O(EF*)，另有 O(V+E) 初始化。這是偽多項式界，因 F* 的數值可對容量輸入位數呈指數。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A16 | 選路可使常數大小網路跑 2U 輪

四點網路 su,ut,sv,vt 容量均為 U，uv 容量1。交替選 s→u→v→t 與 s→v→u→t。每輪δ=1，後一條取消 uv，直到2U輪才達最大值2U。

有理容量可乘公分母化整數，會終止但此界仍非輸入位數多項式。任意實數容量且任意選路的 FF 可能不終止，不能由「每輪增加」推出有限輪，也不能保證非終止序列的極限最優。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A17 | Edmonds–Karp 固定使用 BFS

每輪在正殘餘圖用 BFS 找邊數最少的 s→t 路，不是找容量最大、權重最小或瓶頸最大的路。

六點例可選 s→w→x→t，δ=4；再選 s→y→z→t，δ=7；兩輪值11。這與前四輪 FF 的合法路徑選擇不同。保留反向邊仍是必要的通用實作規則。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A18 | BFS 距離不下降的完整理由

令 d,d′ 是一輪前後從s的邊數距離，不可達為∞。反設有點距離下降，沿新最短路選第一個下降點v及前驅u，故 $d(u) \le d'(u) = d'(v) - 1$。

若 (u,v) 舊已存在，$d(v) \le d(u) + 1 \le d'(v)$，矛盾。若舊不存在，它只能是本輪最短增廣路上 (v,u) 新生成的反向邊；因此 $d(u) = d(v) + 1$，推出 $d(v) = d(u) - 1 \le d'(v) - 2$，仍矛盾。故所有距離不下降，變為∞後也不會重新可達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A19 | 關鍵邊的再次飽和至少升兩層

一輪至少飽和一條關鍵殘餘邊 (u,v)，其 r(u,v)=δ 且 d(v)=d(u)+1。飽和後它消失。

在它重新飽和前，必先沿 (v,u) 增廣使它再出現。此時 d_new(u)=d_new(v)+1≥d_old(v)+1=d_old(u)+2。距離單調確保之後也不下降。能出現在增廣路上的有限距離至多 V−1，故每個方向／edge_id 只有 O(V) 次關鍵事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A09-A20 | 將事件數換成完整時間界

殘餘邊方向至多2E，每輪至少一個關鍵事件，每邊 O(V) 次，總增廣 O(VE)。

每輪 BFS 與路徑更新 O(V+E)。先移除孤立點或用可達子圖使相關 V=O(E)，得到每輪O(E)，總 $O(VE^2)$，外加O(V+E)初始化。容量大小不影響輪次界，在精確實數算術模型也成立；位元運算成本另計。

停止時回傳可行流與可達割。這兩個物件提供獨立於選路歷程的最優驗收。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P01｜暫存流量，再恢復守恆

Ford–Fulkerson每次維持合法flow；push-relabel允許中間點暫存excess。局部push只看一條殘餘邊，relabel提高一個點。

本節以CLRS第3版§26.4–26.5為準。note掃描p38提供逆流取消與殘餘容量的前置概念，不把它當作push-relabel的來源。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P02｜preflow與excess的精確語義

對原圖每邊維持0≤f(u,v)≤c(u,v)。令e(u)=Σ_v f(v,u)−Σ_v f(u,v)。preflow要求所有u≠s有e(u)≥0。active只指u∈{s,t}且e(u)>0。

正向殘餘容量為c−f，反向為f。本例無反平行原邊，反向push直接減少對應原邊flow。source excess可以為負；sink雖可有正excess，永不discharge。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P03｜height不是即時最短距離

合法height滿足h(s)=n、h(t)=0，且每條殘餘邊(u,v)都滿足h(u)≤h(v)+1。它是證明用標籤，不保證等於到t的距離。

若存在簡單殘餘s→t路徑，長度k≤n−1。沿路相加得n=h(s)≤h(t)+k=k，矛盾。因此合法height下始終沒有殘餘s→t路徑。此時preflow仍可能不守恆，不能直接宣告完成。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P04｜初始化先飽和source出邊

設所有flow、excess與height為0，改h(s)=n，將每條(s,v)飽和並更新e(s)、e(v)。這個步驟是初始化，不受一般PUSH的height差1條件限制。

source正向邊已無殘餘容量；反向邊v→s符合0≤n+1，其他殘餘邊兩端height為0。因此同時建立preflow與合法height。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P05｜push的適用條件與更新

PUSH(u,v)要求u active、$c_f(u,v)>0$且$h(u)=h(v)+1$。設$\delta=\min(e(u),c_f(u,v))$，更新原邊flow或取消逆向flow，再令$e(u)-=\delta$、$e(v)+=\delta$。

δ不超過excess與容量，故preflow保持。新出現的逆邊$v\to u$滿足$h(v)=h(u)-1\le h(u)+1$。飽和push使該殘餘邊消失；非飽和push使$e(u)=0$。兩者同時成立時歸類為飽和。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P06｜沒有合法出邊才relabel

RELABEL(u)要求u active，且所有殘餘鄰居v都有$h(u)\le h(v)$，等價於沒有admissible出邊。設$h(u)=1+\min\{h(v):c_f(u,v)>0\}$。

excess>0意味存在正流入邊，其逆邊提供至少一個殘餘鄰居，所以min不是空集合。新height嚴格增加；所有出邊仍合法，入邊右側height變大只會放鬆限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P07｜generic正確性與終止分開證明

只要存在active點，若有admissible邊就push，否則relabel適用。故停止時所有非s/t點excess為0，preflow成為flow。合法height排除殘餘s→t路徑，由max-flow min-cut得最大流。

這是「若終止則正確」。後面的操作次數上界才保證必定終止。證明不依容量是否整數，也不以容量值當迭代上界。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P08｜height界與relabel成本

任一active u都有殘餘路徑回s。反證可取由u可達集合S，若s∉S且無殘餘出邊，跨入S的原flow必為0，而Σ_{v∈S}e(v)≤0，與u active矛盾。

取簡單u→s路徑，長≤n−1，得h(u)≤h(s)+n−1=2n−1。height單調且只對active點relabel，因此每點O(n)次、總O(n²)次。每次掃N(u)求min需O(deg(u))，全部relabel為Σ_u O(n deg(u))=O(nm)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P09｜飽和push為何只有O(nm)

一次u→v飽和後，再次同向飽和前必先有v→u的push補回殘餘容量。第一次h(u)=h(v)+1，反推時h′(v)=h′(u)+1≥h(u)+1。

再次u→v可push時h″(u)=h″(v)+1≥h(u)+2。因此每方向重複飽和會強迫height至少增加2，每個殘餘方向O(n)次。原圖m條邊產生至多2m個方向，合計O(nm)。每次push本身O(1)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P10｜非飽和push的位能帳

令Φ=Σ_{u active}h(u)，初值0且非負。非飽和push移除u，至多新增v。因h(u)=h(v)+1，Φ至少降1；若v為s/t則不加入。

一次relabel或飽和push使Φ增加至多O(n)。relabel $O(n^2)$次，飽和push O(nm)次，故非飽和push $O(n^3+n^2m)$。刪除與s/t無關的孤立成分後n=O(m)，得到$O(n^2m)$。容量大小不在此上界中。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P11｜從操作數轉為實際時間

不能把「有$O(n^2m)$次操作」直接當時間證明。以雙向殘餘arc配對實作$O(1)$的push；保存excess與active容器以$O(1)$選active點。

每點保留current arc，掃過整個$N(u)$才relabel並重設。每點$O(n)$輪，總掃描與relabel成本$O(nm)$。active點可任選並discharge，所以仍是generic的合法排程。加上push次數得到$O(n^2m)$，初始化$O(n+m)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P12｜relabel-to-front保存所有中間點

L是$V-\{s,t\}$全部頂點的鏈結表，包含$e=0$的點。$N(u)$包含原圖所有入邊與出邊的另一端，固定順序，不隨殘餘容量刪除。每點有跨呼叫保留的current指標。

DISCHARGE(u)重複至$e(u)=0$。current=NIL時relabel並重設表首；current admissible時push且不前進；其餘情形current移到下一鄰居。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P13｜前移規則與current arc的合法性

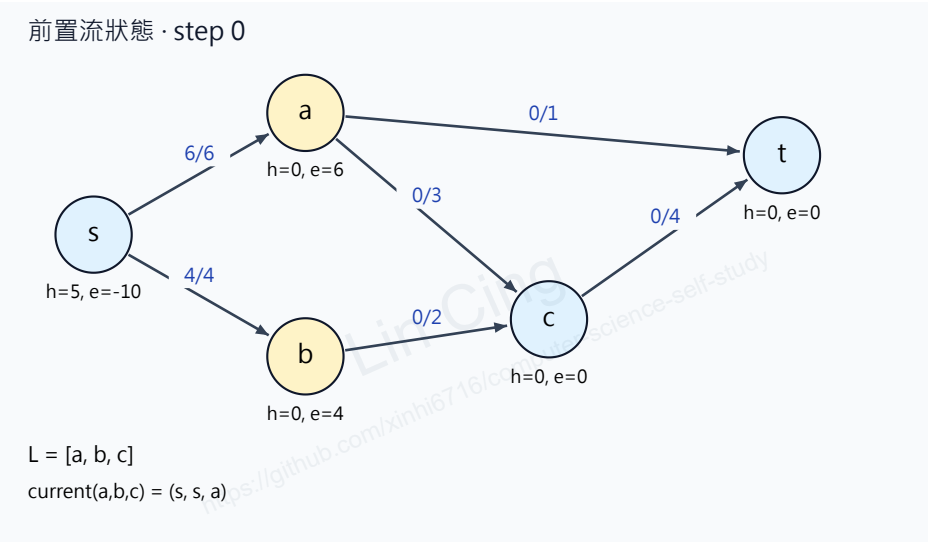
記old=h(u)，DISCHARGE(u)後若h(u)>old，才把u移到L表首。下一個u必為移動後的u.next。未升高時照原順序前進；不把零excess點刪除。

current跨呼叫保留為何安全？push不創造admissible邊；別點relabel後沒有admissible入邊。故u尚未relabel的一輪內，已跳過的u出邊不會變可用。掃到NIL才可斷定relabel適用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P14｜固定五點例與初態

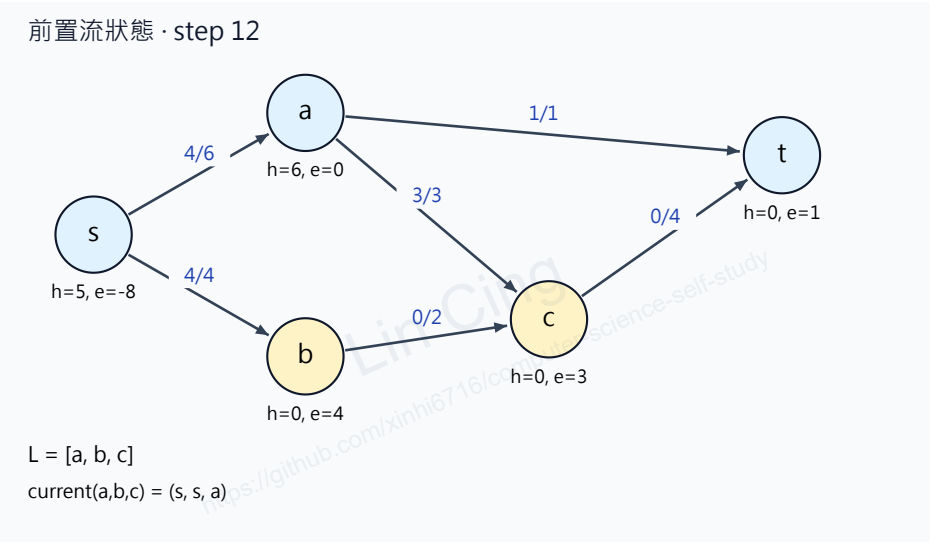
V=(s,a,b,c,t)，n=5。原邊容量為sa=6、sb=4、ac=3、at=1、bc=2、ct=4。固定N(a)=[s,c,t]、N(b)=[s,c]、N(c)=[a,b,t]，L=[a,b,c]。



step 0的h=(5,0,0,0,0)、e=(-10,6,4,0,0)、current=(s,s,a)、L=[a,b,c]。

P15｜a先送出，再退還source

a在height 1送3給c、送1給t，仍有excess 2。此時只有a→s殘餘出邊，relabel到6後退2給s。source高度固定5，故退流的height差仍為1。

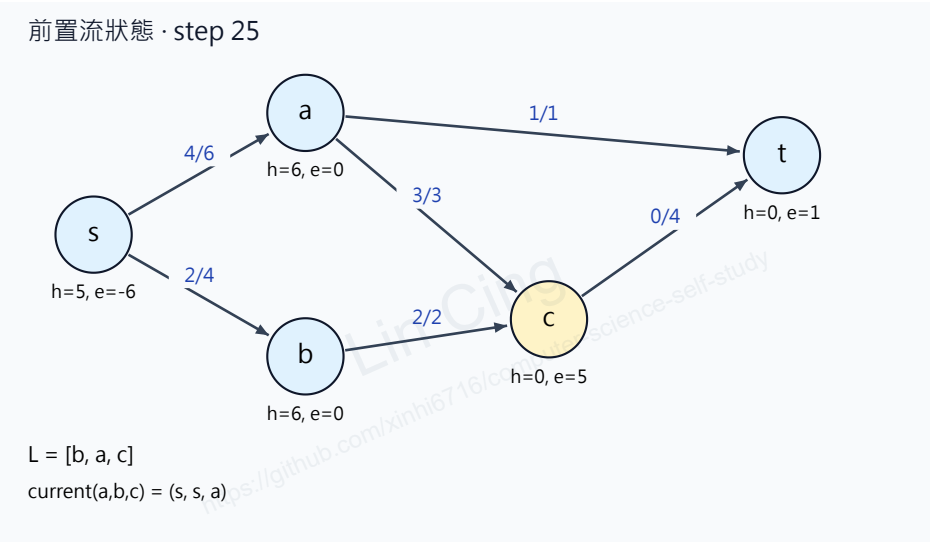


P15｜a先送出，再退還source （續）

step	operation	h(s,a,b,c,t)	e(s,a,b,c,t)	current(a,b,c)	L
5	relabel並重設arc a	5,1,0,0,0	-10,6,4,0,0	s,s,a	a,b,c
7	push a a→c δ=3	5,1,0,0,0	-10,3,4,3,0	c,s,a	a,b,c
9	push a a→t δ=1	5,1,0,0,0	-10,2,4,3,1	t,s,a	a,b,c
11	relabel並重設arc a	5,6,0,0,0	-10,2,4,3,1	s,s,a	a,b,c
12	push a a→s δ=2	5,6,0,0,0	-8,0,4,3,1	s,s,a	a,b,c
14	移至表首 a	5,6,0,0,0	-8,0,4,3,1	s,s,a	a,b,c

P16｜b前移，c再把多餘量退給a

b送2給c、退2給s後前移，L=[b,a,c]。a的零excess discharge立即返回，之後處理c。c先送4給t，再升到7退1給a；c前移後L=[c,b,a]。



P16｜b前移，c再把多餘量退給a （續）

step	operation	h(s,a,b,c,t)	e(s,a,b,c,t)	current(a,b,c)	L
18	relabel並重設arc b	5,6,1,0,0	-8,0,4,3,1	s,s,a	a,b,c
20	push b b→c δ=2	5,6,1,0,0	-8,0,2,5,1	s,c,a	a,b,c
22	relabel並重設arc b	5,6,6,0,0	-8,0,2,5,1	s,s,a	a,b,c
23	push b b→s δ=2	5,6,6,0,0	-6,0,0,5,1	s,s,a	a,b,c
25	移至表首 b	5,6,6,0,0	-6,0,0,5,1	s,s,a	b,a,c
32	relabel並重設arc c	5,6,6,1,0	-6,0,0,5,1	s,s,a	b,a,c
35	push c c→t δ=4	5,6,6,1,0	-6,0,0,1,5	s,s,t	b,a,c
37	relabel並重設arc c	5,6,6,7,0	-6,0,0,1,5	s,s,a	b,a,c
38	push c c→a δ=1	5,6,6,7,0	-6,1,0,0,5	s,s,a	b,a,c
40	移至表首 c	5,6,6,7,0	-6,1,0,0,5	s,s,a	c,b,a

P17｜終態、割證書與完整操作表

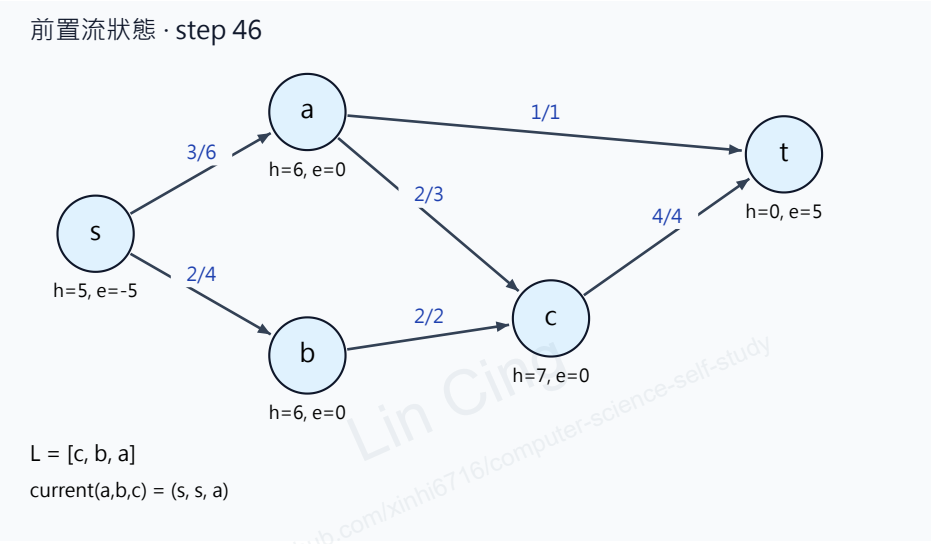
a保留current=s，以height 6再退1給s。最後e=(-5,0,0,0,5)，流值5。割S={s,a,b,c}、T={t}容量at+ct=1+4=5，與流值相等。

本章詳解附47列完整表，含每次arc前進、discharge起訖與list前移。圖中step直接對應同編號表列。

step	operation	h(s,a,b,c,t)	e(s,a,b,c,t)	current(a,b,c)	L
44	push a a→s δ=1	5,6,6,7,0	-5,0,0,0,5	s,s,a	c,b,a
46	終止	5,6,6,7,0	-5,0,0,0,5	s,s,a	c,b,a

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

P17｜終態流量與守恆



終態邊流為sa=3、sb=2、ac=2、at=1、bc=2、ct=4；各中間點的入流等於出流。

P18｜relabel-to-front為何是$O(n^3)$

外層迴圈每次測試時，L是中間點admissible network的拓樸序，且目前u之前皆為零excess。邊的height降1，故圖無環。初態所有中間點height相同，且u在表首。

若discharge沒有relabel，push不創造admissible邊，因此不會送至u之前；u清空後前進，兩項仍成立。若u曾relabel，最後u無admissible入邊，且push不創造此邊；移u至表首後，原有其他點的邊順序不變，u已清空，下一點之前只有u，兩項恢復。掃到表尾時全體中間點excess為0；結合合法height排除增廣路徑，得到最大流。

這些性質不要求在discharge中成立：step 37時c→a admissible但c在a之後；step 38時a的excess為1；step 40將c前移後恢復。

相鄰relabel間至多O(n)次DISCHARGE，共$O(n^2)$次relabel，故$O(n^3)$次呼叫。relabel、arc前進及飽和push共O(nm)；每次DISCHARGE至多一個非飽和push，因它清空u。總時間$O(n^3+nm)=O(n^3)$，簡單圖有$m=O(n^2)$。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

把需求寫成流的限制

本節用非負弧流 x(e) 表示原始有向邊上的運量，$0 \le x(e) \le c(e)$。若主章用反對稱淨流 f，則 $f(u,v)=x(u,v)-x(v,u)$。帶下界時必須保留每條弧的 x，不能只用淨流抹去兩方向的義務。

每次歸約都證明原解能轉成流、目標值相等、流能還原原解。對離散選擇，還要確認整數流。

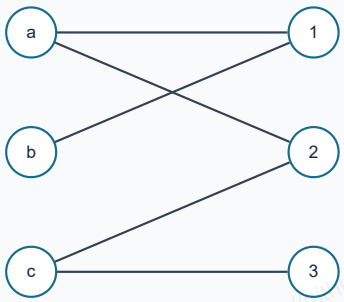
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二分匹配的建網

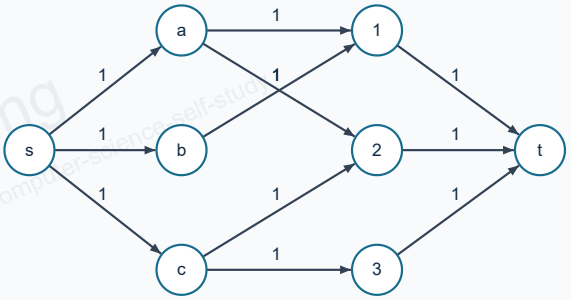
給定 G=(L∪R,E)，L、R 不交且每條邊跨兩側。加新點 s,t，把每條無向邊定向為 L→R，加入 s→u 與 v→t。所有容量為 1。

例 L={a,b,c}，R={1,2,3}，E={a1,a2,b1,c2,c3}。轉換後有 8 點、11 弧。圖左為原問題，圖右為容量網。

原二分圖



轉換網路，全部容量 1



解 {a2,b1,c3}，流值 3，割 {s} 容量 3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

匹配到流的映射

取 M={a2,b1,c3}。每條 uv∈M 沿 s→u→v→t 送 1，其餘弧送 0。匹配的端點不重複，所以每條 source 弧與 sink 弧至多使用一次。

內點每次收到 1 就送出 1，容量及守恆都成立。跨 L→R 的總流為 |M|=3，故流值是 3。割 S={s} 的容量為 3，因此這個流及匹配都最優。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

整數流到匹配的映射

給定整數流 x，取 M={uv∈E:x(u,v)=1}。因 s→u 容量為 1，守恆迫使 u 至多一條出弧有正整數流。v→t 同理使每個 v 至多被選一次。故 M 是匹配且 |M|=|x|。

從零流執行 Ford–Fulkerson，整數容量使每次殘餘容量、瓶頸及更新後流都為整數。這證明存在整數最大流，不表示每個最大流都是整數。例 $K_{2,2}$ 的四條中間弧各送 1/2，仍是值 2 的最大流，但正流弧不能直接當匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

匹配失敗的割證書

若 L={a,b,c}、R={1,2,3} 且 E={a1,b1,c2,c3}，a 與 b 只能爭取 1。可取 {a1,c2}，大小 2。

令 S={s,a,b,1}。從 S 到補集只有 s→c 與 1→t，各容量 1，所以割容量 2，最大匹配也是 2。集合 A={a,b} 滿足 |N(A)|=1<2，顯示不能匹配整個 L。不能把「極大」匹配誤當「最大」匹配。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

頂點容量必須限制總通過量

原網 s→v 容量 5、v→t 容量 5，但 v 每單位時間最多處理 3。單獨限制每條鄰邊為 3，不能一般化處理多條入邊。正確限制是 Σ_u x(u,v)≤b(v)。

把 v 拆為 v_in,v_out，以容量 b(v)=3 的內部弧相連。每條入弧改接 v_in，每條出弧從 v_out 出發。例中最大流 3，割 {s,v_in} 的容量為 3。

原網．v 的容量為 3



拆點．以內部弧限制總量



最大流 3。割 {s,v_in} 的容量 3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

拆點的雙向證明與尺寸

原可行流在每條改接弧維持原值，並令內部弧流等於 v 的總流入。守恆及頂點容量保證轉換可行。反過來，v_in 與 v_out 的守恆迫使內部弧等於總流入及總流出；合併兩點就還原原流。

若全部 n 點都拆開，得到 2n 點、m+n 弧。只拆 k 個受限內點則為 n+k 點、m+k 弧。若 source 或 sink 也有總容量，必須拆開並把外部 source 接到 s_in、外部 sink 接在 t_out。

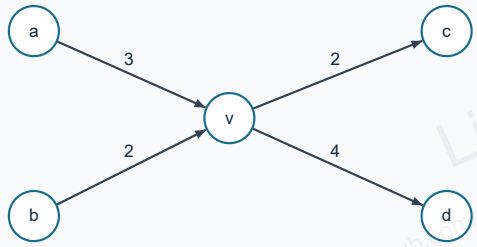
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多源多匯加入超點

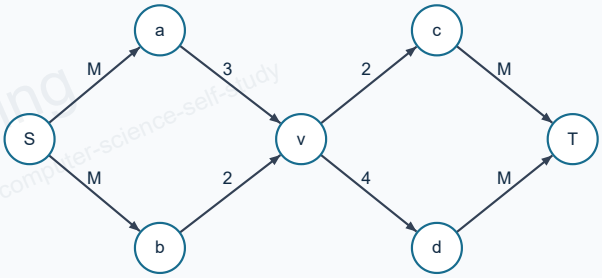
原問題有供應點 a,b、需求點 c,d，中間點 v。弧容量為 a→v=3、b→v=2、v→c=2、v→d=4。源點沒有原始入弧，匯點沒有原始出弧。

加入 S→a,S→b,c→T,d→T。取大容量 M=11，等於原弧容量總和。送 a→v→c 2、a→v→d 1、b→v→d 2，總量 5。割 {S,a,b} 容量 3+2=5，證明最優。

原多源多匯網



超點．人工弧容量 M=11



流值 5．送到 c 為 2、d 為 3。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

總量最大與配額可行不同

原解延伸時令 S→a 等於 a 的淨流出、c→T 等於 c 的淨流入，超點之外便全部守恆。移除超點則還原相同總量。這個證明要求各供應點淨流出非負、需求點淨流入非負。

若 a,b 必須供應 3,2，c,d 必須接收 2,3，新增弧容量分別設為 3,2,2,3，並檢查最大流是否等於 P=5。總新增 source 容量與 sink 容量皆為 5，值 5 等價於每一配額弧飽和。上一例可行。若 c 的配額改為 3、d 改為 2，則 v→c 只有 2；割 {S,a,b,v,d} 容量 2+2=4，配額失敗。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

有限大容量必須有上界依據

最大流模型中，新增弧只需不壓低原問題最佳值。原弧有限非負容量時，M=Σ_e c(e) 是安全但可能寬鬆的選擇。也可用已證明的原網總輸送上界，例如本例 3+2=5。M=4 放在每條超點弧仍足夠此例，但不能憑感覺選常數。

帶需求的輔助最大流只需送 D 單位，故不應限制需求的人工連接弧可取容量 D。若為了用割嚴格排除人工弧，須選 M 大於已知候選割上界，例如整數模型用 U+1。M≥U 足夠保最佳值，未必能排除同值人工割。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

下界先轉成每點不平衡量

可行 circulation 在每個點都守恆，且 ℓ(e)≤x(e)≤u(e)，其中 0≤ℓ≤u。令 y(e)=x(e)−ℓ(e)，新容量為 u−ℓ。定義 d(v)=Σ_in ℓ−Σ_out ℓ。

逐點代入原守恆得到 Σ_in(y+ℓ)=Σ_out(y+ℓ)，因此 Σ_out y−Σ_in y=d(v)。若 d(v)>0，y 必須淨流出 d(v)；若 d(v)<0，y 必須淨流入 −d(v)。每條下界在一個頭點加、尾點減，故 Σ_v d(v)=0。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

超點方向由守恆推導

加新源 S*、新匯 T*。若 d(v)>0，加 S*→v 容量 d(v)；若 d(v)<0，加 v→T* 容量 −d(v)。設 D=Σ_{d>0}d=Σ_{d<0}(−d)。

當需求弧飽和，d>0 的點滿足 Σ_in y+d=Σ_out y；d<0 的點滿足 Σ_in y=Σ_out y−d；d=0 的點照常守恆。三種情形都導回 Σ_out y−Σ_in y=d。

可行的充要條件是輔助最大流值 D，等價於所有 S* 出弧飽和。T* 入弧總容量同為 D，所以它們也全部飽和。刪超點並還原 x=ℓ+y。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

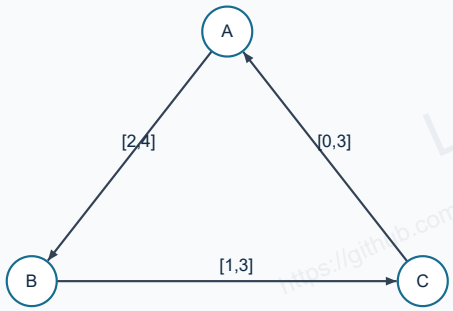
有下界循環的完整可行例

原圖 A→B:[2,4]、B→C:[1,3]、C→A:[0,3]。方括號為 [ℓ,u]。d(A)=0−2=−2，d(B)=2−1=1，d(C)=1−0=1，D=2。

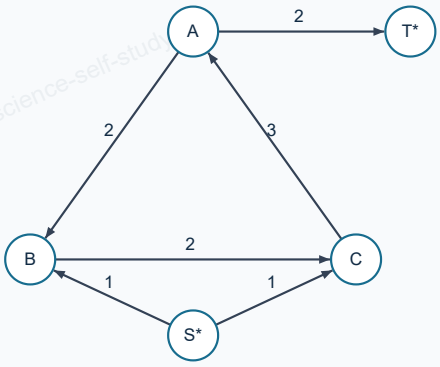
新容量依序是 2,2,3，並加 S*→B=1、S*→C=1、A→T*=2。沿 S*→B→C→A→T* 送 1，再沿 S*→C→A→T* 送 1。得到 y=(0,1,2)，還原 x=(2,2,2)。

A、B、C 各入 2、出 2，且三條弧均符合上下界。

原 circulation · 下界與上界



輔助網 · 標示容量 u−ℓ



y=(0,1,2) · x=(2,2,2) · 所有需求弧飽和。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

不可行例要展示不足的割

保持上一例，但把 C→A 改成 [0,1]。d 與 D 不變。S*→C→A→T* 可送 1，之後 C→A 飽和。

取輔助割 S={S*,B,C}，補集為 {A,T*}。唯一出割弧 C→A 容量 1，故最大流至多 1<D=2。原網也能直接看出 A 至少必須送出 2，卻最多只能收到 1，違反守恆。不能以「所有剩餘容量非負」判斷下界可行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從帶下界 s-t 流到 circulation

先限於要求流值非負的 s-t 流。加回弧 t→s，範圍 [0,M]，使所有點都守恆；回弧流量等於原 s-t 流值。若要測試恰好 F，改用 [F,F]，再套前述需求歸約。

例 s→a:[1,3]、a→t:[1,2]，要求 F=2。加入 t→s:[2,2]，得到 d(s)=1、d(a)=0、d(t)=−1。輔助路 S*→s→a→t→T* 送 1，原兩弧還原為 2，回弧為 2。若 F=3，a→t 上界 2，無可行解。

只找可行流值時，可用 M=Σ 原弧上界。若問題允許負流值，單一非負回弧未涵蓋該情況，必須另建模型。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

還原答案時逐項驗收

先核對原問題的選擇或配額，再核對每條原弧的上下界與每點守恆。最後給出等值割、需求弧飽和，或失敗割作證書。整數性只在所有轉換後容量為整數且求得整數流時使用。

一般實數容量的可行流可以是分數。即使所有容量為整數，任意可行流或任意最大流也可能分數。匹配讀出必須選整數最大流。帶整數上下界的可行 circulation 可透過整數輔助流還原成整數解。

來源對照採 CLRS 26.1、26.3 與習題 26.1-7、26.2-5、26.2-6、26.3-2。下界歸約及本節數例為自行推導，check.py 以枚舉及割逐一核對。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd ed., Chapter 26, Maximum Flow. Original handwritten algorithms notes (演算法_wjungle), network-flow pages. MIT 6.046J, Fall 2004, Lectures 20 and 21, Maximum Flow and Edmonds–Karp.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study